TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE VERACRUZ

ING. EN SISTEMAS COMPUTACIONALES

ADMINISTRACIÓN DE BASES DE DATOS

Grupo:

6J3

ACTIVIDAD:

Análisis del entorno de MongoDB Atlas: servicios, arquitectura y administración en la nube y creación de un clúster

ALUMNO:

Ramírez Sánchez Luis Eduardo

NÚMERO DE CONTROL:

22020500

¿Qué es MongoDB Atlas?

MongoDB Atlas es un servicio de base de datos administrada en la nube, del tipo **DBaaS** (Database as a Service). MongoDB se encarga de instalar, configurar, actualizar, respaldar y escalar los servidores, mientras que el usuario solo trabaja con sus datos. Atlas funciona sobre los tres proveedores de nube principales: **Amazon Web Services (AWS)**, **Google Cloud** y **Microsoft Azure**, en decenas de regiones del mundo.

MongoDB es una base de datos **NoSQL orientada a documentos**: en lugar de tablas con filas y columnas fijas, guarda documentos con formato parecido a JSON (internamente en **BSON**, JSON binario), y cada documento puede tener campos distintos. La siguiente tabla compara sus conceptos con los de una base relacional como MySQL:

MongoDB	MySQL (relacional)	Descripción
Cluster	Servidor	Conjunto de servidores donde vive la base de datos
Base de datos	Base de datos	Agrupa los datos de una aplicación
Colección	Tabla	Grupo de documentos, sin esquema fijo
Documento	Fila (registro)	Un registro en formato tipo JSON
Campo	Columna	Un dato dentro del documento
_id	Llave primaria	Identificador único, se genera solo si no se indica

¿Qué se puede hacer en Atlas?

- **Crear clusters** en minutos desde la página web, la terminal (Atlas CLI), la API REST o herramientas de infraestructura como código (Terraform).
- **Elegir dónde viven los datos:** proveedor de nube y región, o varias regiones a la vez.
- **Escalar** el cluster (más memoria, CPU o disco) sin apagarlo, o activar el autoescalado.
- **Administrar la seguridad:** usuarios de base de datos, roles, direcciones IP permitidas y cifrado.
- **Explorar y editar datos** desde el navegador con el Data Explorer, sin instalar nada.
- **Conectar aplicaciones** mediante drivers oficiales para Python, Java, Node.js, C#, Go y otros lenguajes.
- **Monitorear el rendimiento** con métricas y alertas.
- **Respaldar y restaurar** información, y **migrar** bases existentes hacia Atlas, incluso desde bases relacionales.

Servicios que contiene la nube

Categoría	Servicio	Descripción
Base de datos	Atlas Database	Servicio principal: MongoDB administrado en replica sets con alta disponibilidad y failover automático.
Base de datos	Global Clusters	Datos distribuidos por regiones del mundo para reducir latencia o cumplir leyes locales.
Búsqueda e IA	Atlas Search	Búsqueda de texto completo dentro de los datos, con tolerancia a errores y orden por relevancia.
Búsqueda e IA	Vector Search	Búsqueda por significado usando embeddings; base para aplicaciones con IA (RAG, chatbots).
Búsqueda e IA	AI Model APIs / Agent Engine	Modelos de Voyage AI para generar embeddings y servicio para desplegar agentes de IA.
Datos en tiempo real	Stream Processing	Procesa flujos de datos (sensores, eventos) antes de guardarlos.
Datos en tiempo real	Triggers	Funciones que se ejecutan automáticamente al cambiar un dato o en un horario.
Análisis	Atlas SQL	Permite consultar los datos con SQL desde herramientas como Power BI o Tableau.
Análisis	Data Federation	Consulta datos de varias fuentes (cluster, archivos en Amazon S3) como si fueran una sola base.
Análisis	Charts / Visualization	Gráficas y dashboards creados directamente con los datos.
Operación	Backup	Respaldos automáticos y restauración a un punto en el tiempo (tiers de pago).
Operación	Monitoring y alertas	Métricas de conexiones, operaciones, red y disco; Performance Advisor sugiere índices.
Operación	Migration	Herramientas para mover bases existentes hacia Atlas.
Seguridad	Database & Network Access	Usuarios, roles, lista de IPs permitidas, cifrado y auditoría.
Administración	Billing	Facturación, créditos, facturas y explorador de costos.

Tipos de cluster

Al crear un cluster, Atlas ofrece tres niveles principales (tiers). Los valores se tomaron de la pantalla de creación del cluster:

Característica	Free (M0)	Flex	Dedicated (M10)
Uso	Aprender y explorar	Desarrollo y pruebas	Desarrollo y apps con poco tráfico
Almacenamiento	512 MB	5 GB	10 GB
RAM / vCPU	Compartidas	Compartidas	2 GB / 2 vCPUs
Conexiones máximas	500	500	1500
Precio	Gratis	Desde $0.011/hora (máx. $30/mes)	$0.08/hora

El plan gratuito tiene limitaciones: solo se permite **un cluster M0 por proyecto**, no se puede pausar manualmente, no incluye respaldos ni todas las métricas, y se pausa solo después de 60 días sin uso. Arriba de M10 están los tiers dedicados de producción (M20, M30, etc.).

Arquitectura: cómo funciona Atlas sobre la nube

Atlas no tiene centros de datos propios; funciona **encima** del proveedor de nube elegido. Se pueden distinguir dos capas:

- **Plano de control (Atlas):** la página web, la API, la facturación y la automatización. Cuando se crea un cluster, Atlas le pide al proveedor las máquinas virtuales y los discos, instala MongoDB y lo configura. El usuario nunca interactúa directamente con AWS.
- **Plano de datos (AWS, Google Cloud o Azure):** donde realmente viven los datos. Los servidores se crean en una cuenta del proveedor que pertenece a MongoDB; por eso no se necesita cuenta de AWS propia.

Dentro del proveedor, cada cluster se ubica en una **región** y cada uno de sus nodos se coloca en una **zona de disponibilidad** distinta, es decir, en centros de datos separados con energía y red independientes. Así, aunque falle un edificio completo, los demás nodos siguen funcionando. Cada nodo es una máquina virtual (EC2 en AWS) con su disco, donde corre el proceso `mongod` y un agente de automatización que recibe órdenes de Atlas.

El cluster gratuito M0 es un caso especial: no tiene máquinas propias, sino que comparte un replica set con otros usuarios (modelo **multi-tenant**). Por eso, al declararlo en un archivo de configuración, el proveedor aparece como `TENANT` y AWS como `backingProviderName`.

Para elegir proveedor y región se considera: dónde corre la aplicación (menos latencia y sin costo de tráfico entre nubes), dónde están los usuarios, las leyes sobre dónde deben guardarse ciertos datos, el costo de cada región y la posibilidad de conectar la red privada del proveedor con Atlas (VPC Peering o PrivateLink).

Nodos y replica set

Un **nodo** es un servidor que ejecuta el proceso de MongoDB (`mongod`). El cluster de esta práctica es un **replica set de tres nodos**: tres servidores con una copia de los mismos datos. Internamente se llama `atlas-54gx2f-shard-0`, como se observa en el prompt de mongosh.

- **Primario:** el único nodo que acepta escrituras (inserciones, actualizaciones y borrados).
- **Secundarios:** copian todo lo que hace el primario y pueden atender lecturas si se configura.

El primario anota cada cambio en un registro especial llamado **oplog** (operations log); los secundarios lo leen y repiten las mismas operaciones en orden, manteniéndose sincronizados casi en tiempo real.

Los nodos se envían **heartbeats** cada 2 segundos. Si el primario deja de responder durante unos 10 segundos, los secundarios realizan una **elección**: gana el nodo que obtiene la **mayoría de votos** y tiene los datos más actualizados, y se convierte en el nuevo primario. Las aplicaciones se reconectan solas (**failover automático**). El protocolo está basado en el algoritmo de consenso **Raft**.

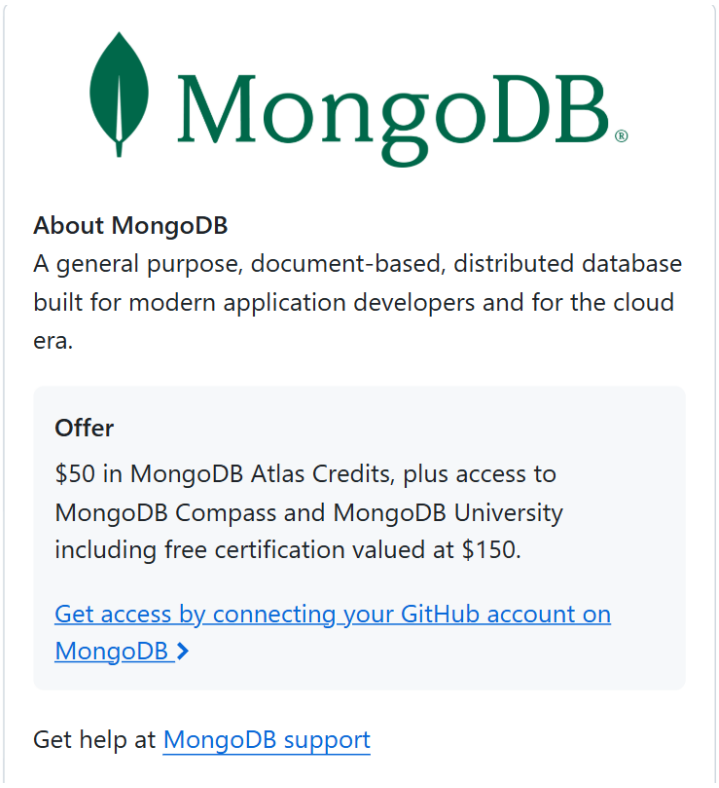
Cluster en la nube vs. cluster local

Aspecto	Nube (Atlas)	Local (self-managed)
Instalación	Minutos, desde la web o un comando	Instalar y configurar cada servidor y el replica set
Hardware	Lo pone el proveedor	Se compra y mantiene
Alta disponibilidad	3 nodos en zonas distintas, failover automático	Se configura manualmente (rs.initiate, rs.add)
Actualizaciones y backups	Automáticos	Responsabilidad propia (mongodump, snapshots)
Seguridad	TLS, lista de IPs y cifrado activados	Hay que activar autenticación, TLS y firewall
Costo	Pago por uso, sin inversión inicial	Inversión en equipo, energía y personal
Control	Limitado (sin acceso al sistema operativo)	Total
Dependencia	Requiere internet y depende del proveedor	Funciona en red propia sin internet

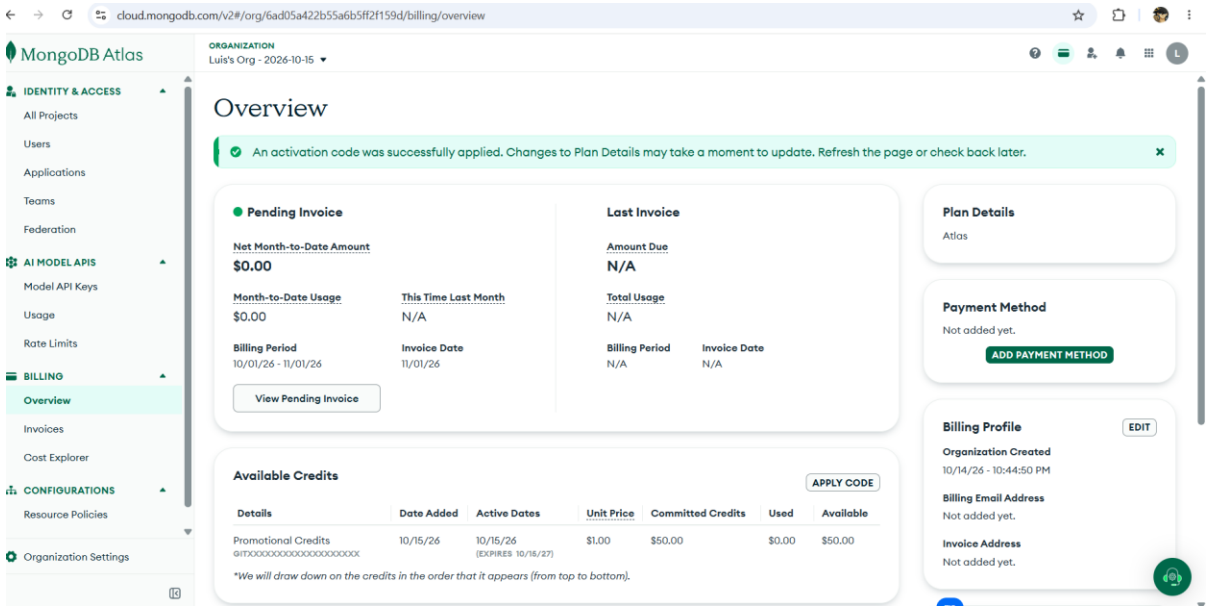
Se aplica el **modelo de responsabilidad compartida**: en la nube el proveedor se encarga de la infraestructura y el usuario de sus datos, usuarios y accesos; en un cluster local todo es responsabilidad propia.

Desarrollo de la práctica

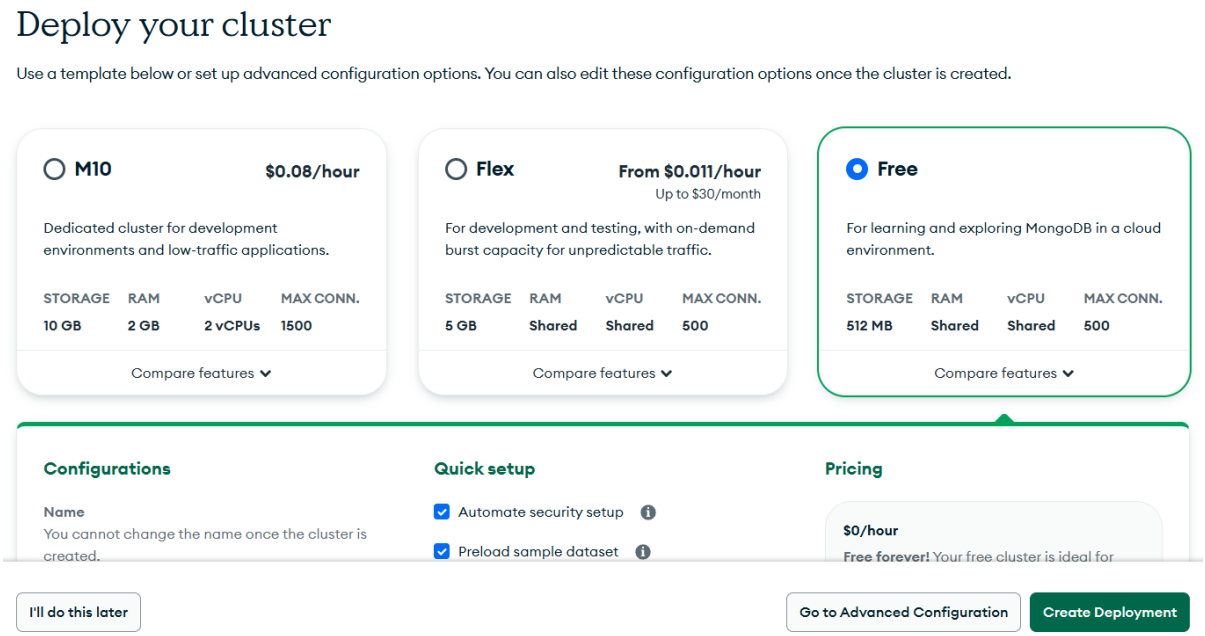
1.-Se reclamó el beneficio de MongoDB dentro del GitHub Student Developer Pack, que incluye $50 en créditos de Atlas, acceso a MongoDB Compass y a MongoDB University con una certificación gratuita.



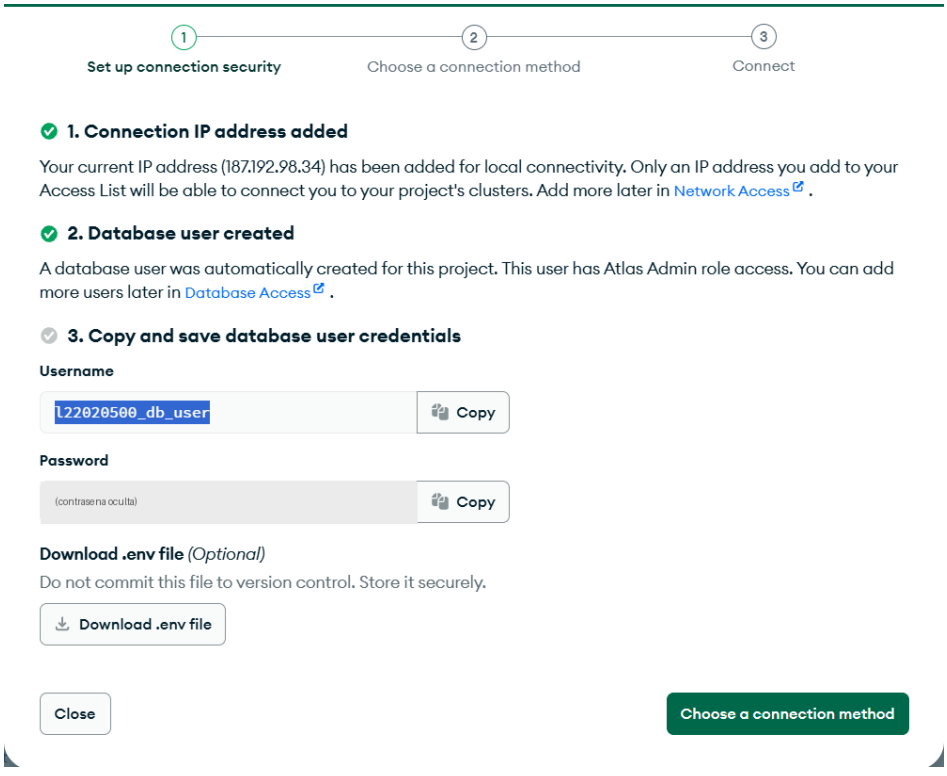
2.-Se aplicó el código de activación en la sección Billing de la organización. Los $50 de créditos promocionales, no fue necesario agregar método de pago.



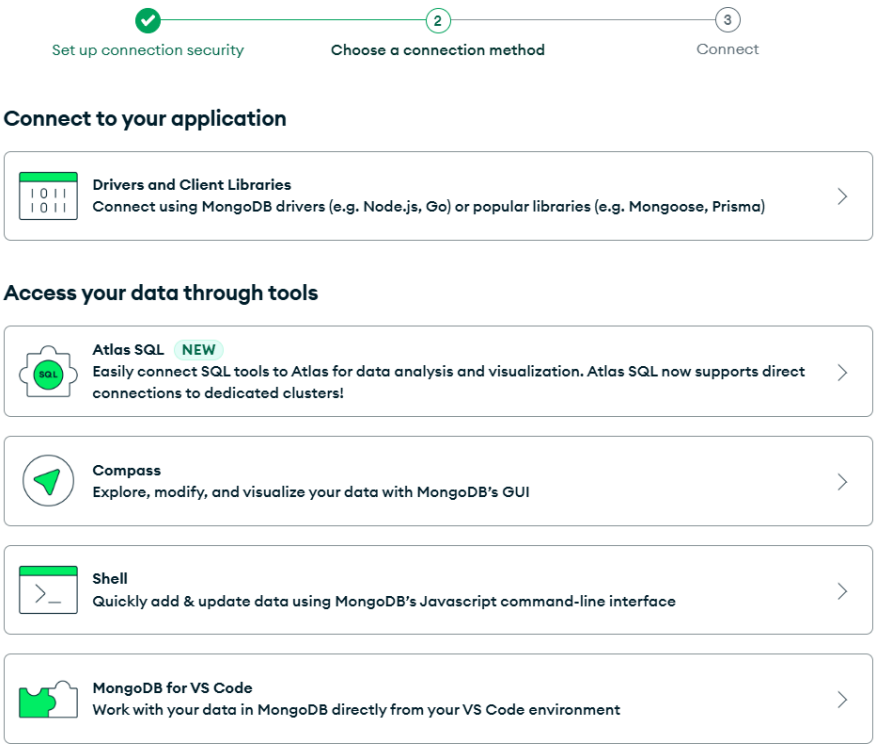
3.-Se creó el clúster eligiendo el plan Free (M0), comparándolo con Flex y M10. Se dejaron activas las opciones de configuración automática de seguridad y de carga del dataset de ejemplo.



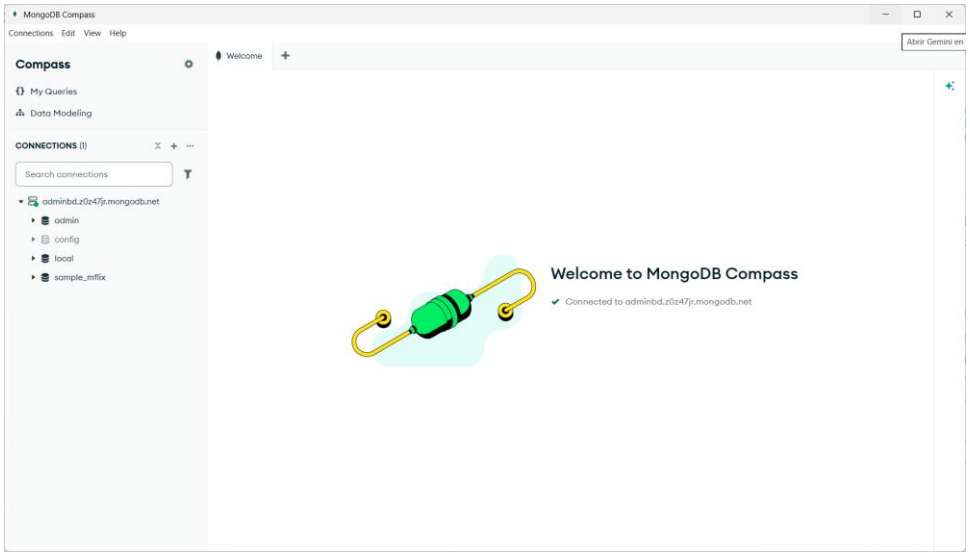
4.-Atlas configuró la seguridad de forma automática: agregó la IP actual a la lista de acceso y creó un usuario de base de datos con rol Atlas Admin. En esta pantalla el usuario no se puede personalizar; se modificó después. (La contraseña se ocultó en la captura).



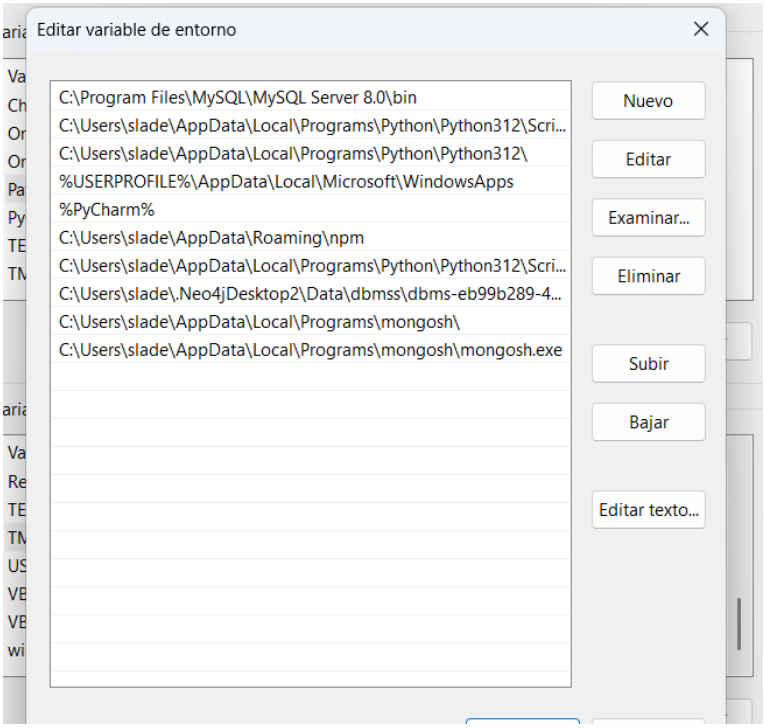
5.-Se revisaron los métodos de conexión que ofrece Atlas: drivers para aplicaciones, Atlas SQL, Compass, Shell (mongosh) y la extensión para VS Code.



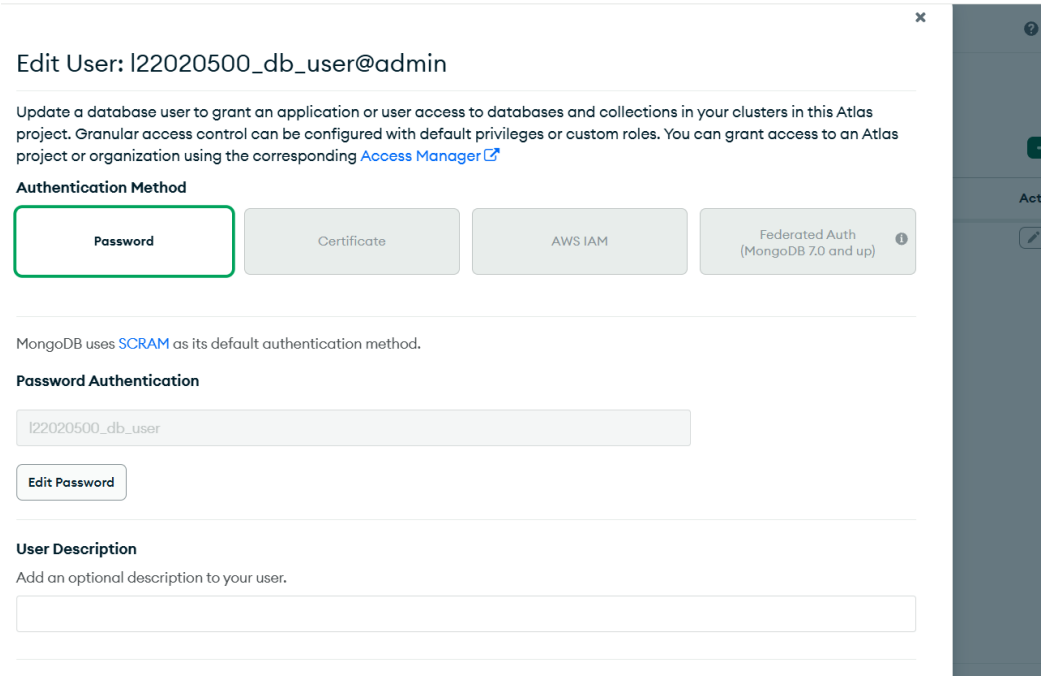
6.-Se conectó MongoDB Compass con el connection string `mongodb+srv://usuario:contraseña@adminbd.z0z47jr.mongodb.net/`. El primer intento falló porque se dejaron los símbolos < > alrededor del usuario, que Compass codificó como %3C y %3E. Al corregirlo se conectó y se muestran las bases admin, config, local y sample_mflix.



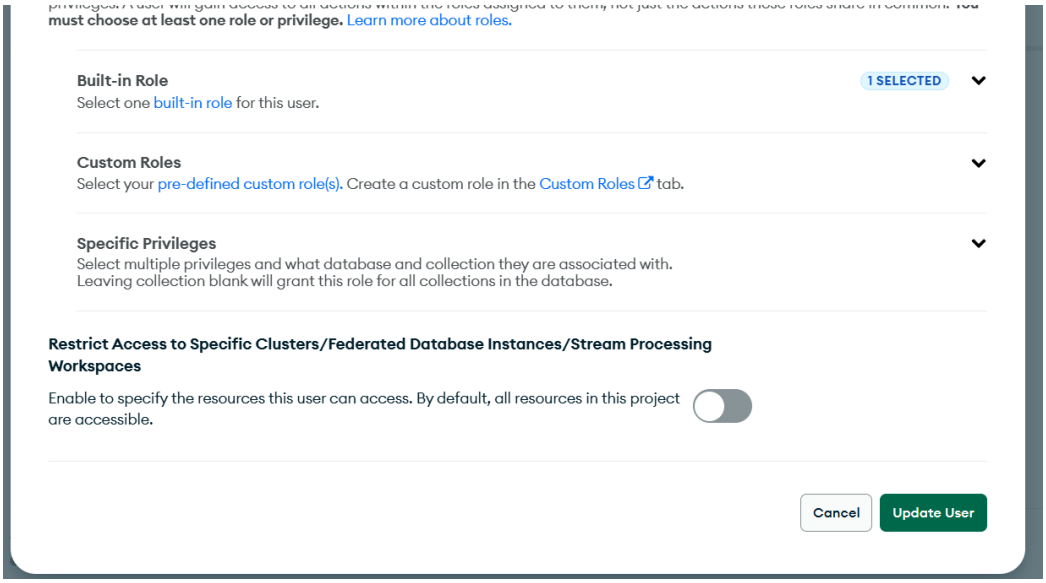
7.-Se instaló mongosh con `winget install MongoDB.Shell` y se revisó la variable de entorno PATH. La entrada correcta es la carpeta `...\Programs\mongosh\`; la que apunta a mongosh.exe se eliminó.



8.-Desde Security ---Database Access se editó el usuario de base de datos. El nombre de usuario no se puede cambiar, solo la contraseña, por lo que se creó un nuevo usuario llamado Eduardo.

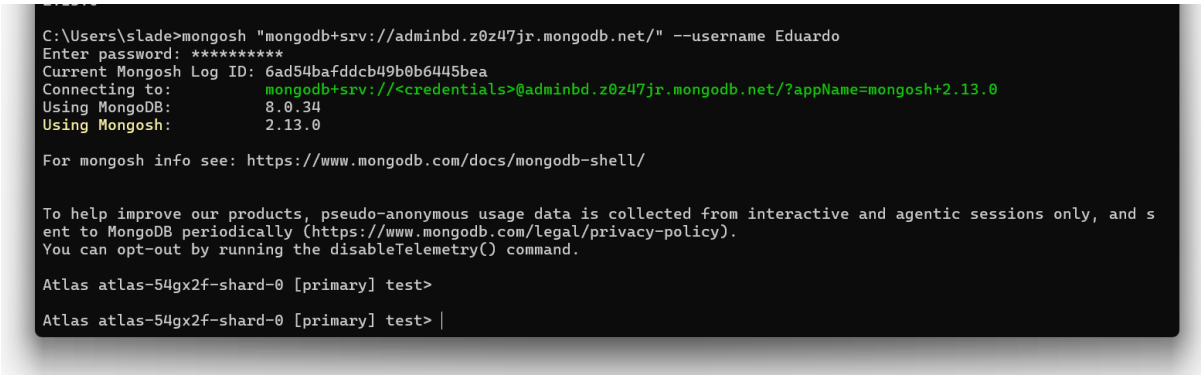


En la parte de privilegios se asigna un rol predefinido, roles personalizados o privilegios específicos, y se puede restringir el acceso del usuario a ciertos clusters del proyecto.

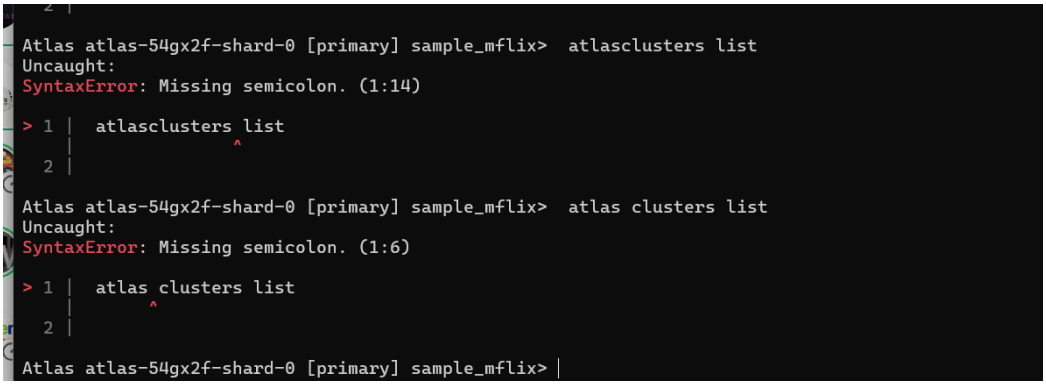


9.-Se conectó por CMD con mongosh usando el nuevo usuario. La contraseña se pide aparte para que no quede en el historial. Se observa la versión del servidor (MongoDB 8.0.34), la de mongosh (2.13.0) y el prompt, que indica el replica set, que el nodo es el [primary] y la base actual (test).

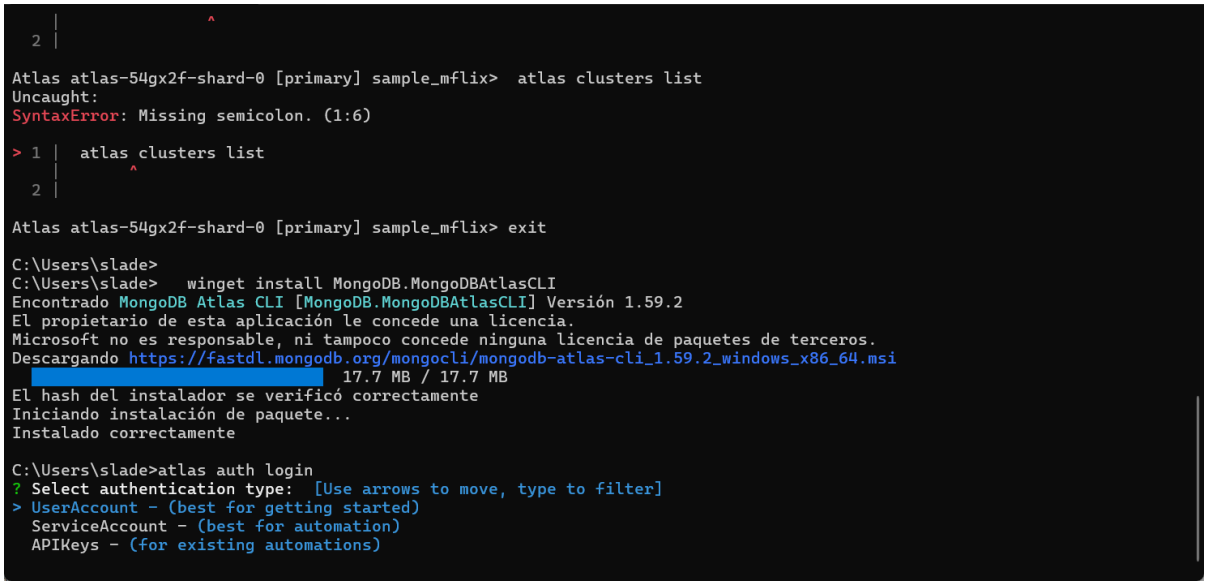
```
mongosh "mongodb+srv://adminbd.z0z47jr.mongodb.net/" --username Eduardo
```



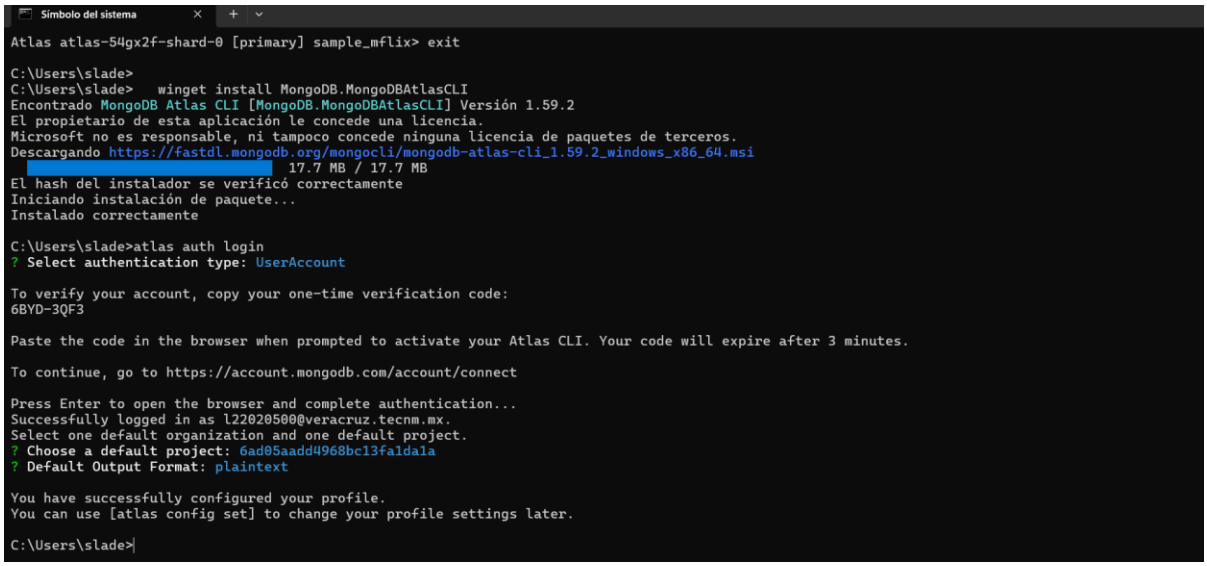
10.-Al intentar ejecutar `atlas clusters list` dentro de mongosh se obtuvo un SyntaxError, porque mongosh solo interpreta JavaScript y sirve para trabajar con los datos; la administración de la infraestructura se hace con otro programa, Atlas CLI.



11.-Se salió de mongosh, se instaló Atlas CLI con `winget install MongoDB.MongoDBAtlasCLI` y se inició sesión con `atlas auth login`, eligiendo UserAccount (cuenta personal). Las otras opciones, ServiceAccount y APIKeys, son para automatización.

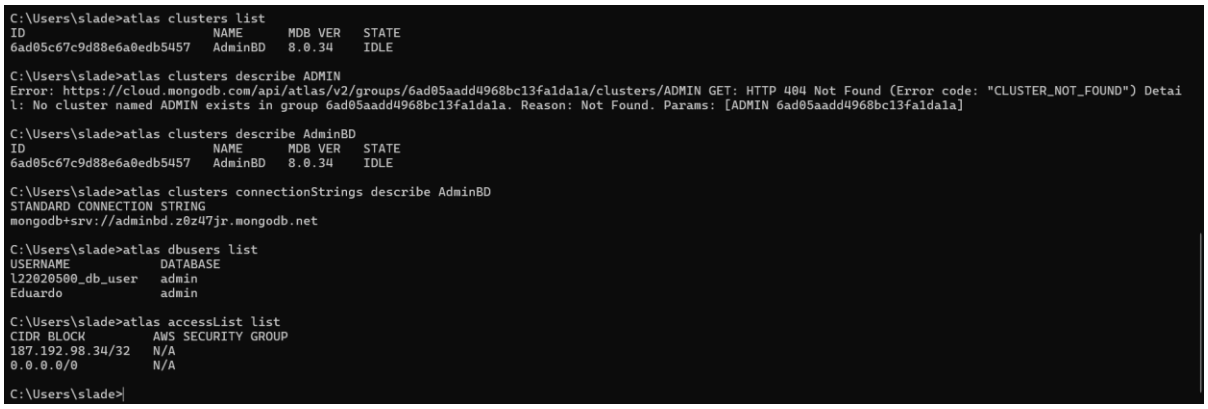


El inicio de sesión se completa con un código de un solo uso que se pega en el navegador. Después se eligen el proyecto por defecto y el formato de salida.



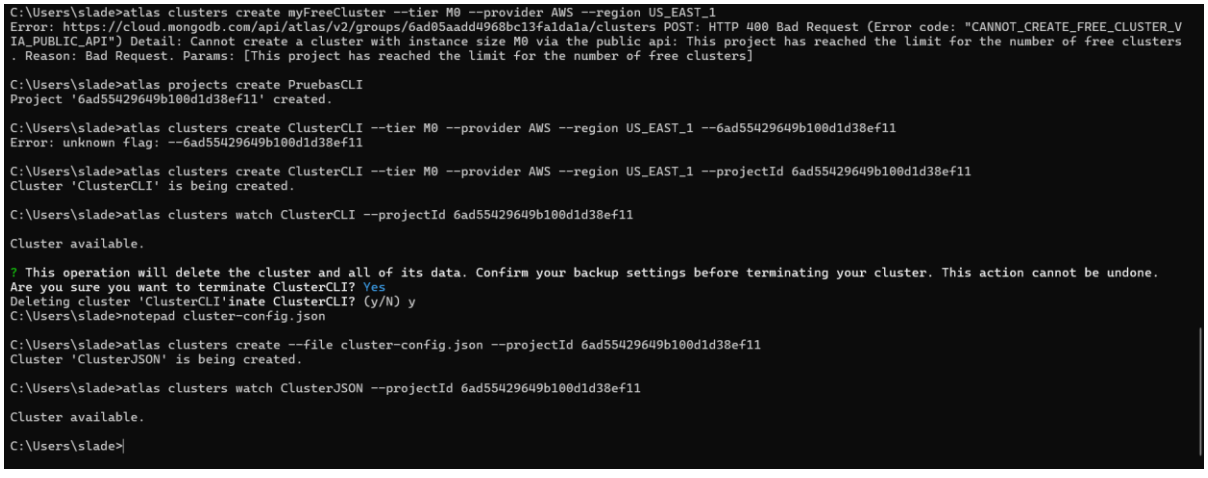
12.-Se consultó la infraestructura desde la terminal. El error 404 al escribir ADMIN muestra que cada comando es en realidad una petición a la API REST de Atlas y que el nombre del cluster debe ser exacto. También se listaron el connection string, los usuarios de base de datos (l22020500_db_user y Eduardo) y la lista de acceso, donde `/32` corresponde a una sola IP y `0.0.0.0/0` a cualquier dirección.

```
atlas clusters list
atlas clusters describe AdminBD
atlas clusters connectionStrings describe AdminBD
atlas dbusers list
atlas accessLists list
```



13.-Al ejecutar el comando completo, Atlas respondió con el error CANNOT_CREATE_FREE_CLUSTER: el proyecto ya tenía su cluster gratuito. Para probar el ciclo de vida se creó el proyecto PruebasCLI y dentro de él el cluster ClusterCLI; con `watch` se esperó hasta que estuvo disponible y después se eliminó con confirmación.

```
atlas projects create PruebasCLI
atlas clusters create ClusterCLI --tier M0 --provider AWS --region US_EAST_1 --
projectId <ID>
atlas clusters watch ClusterCLI --projectId <ID>
atlas clusters delete ClusterCLI --projectId <ID>
```



15.-Finalmente se creó un cluster a partir de un archivo JSON (infraestructura como código), tomando como base la estructura del artículo de referencia. En ella `clusterType` indica el tipo de cluster, `regionConfigs` el proveedor y región, `priority` la preferencia para alojar al primario, `electableSpecs` el tamaño y número de nodos elegibles, y `backupEnabled` los respaldos.



Como el ejemplo usa un tier M30 de pago, se adaptó al tier gratuito, donde el proveedor es TENANT (servidor compartido). El cluster ClusterJSON se creó correctamente, como se ve en las últimas líneas de la captura del paso anterior.

```
{
  "name": "ClusterJSON",
  "clusterType": "REPLICASET",
  "replicationSpecs": [{
    "regionConfigs": [{
      "providerName": "TENANT",
      "backingProviderName": "AWS",
      "regionName": "US_EAST_1",
      "priority": 7,
      "electableSpecs": { "instanceSize": "M0" }
    }]
  }]
}

atlas clusters create --file cluster-config.json --projectId <ID>
```